MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 18 — Fichiers et I/O

Lire, écrire, persister — dialoguer avec le système de fichiers

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

16 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 18 — Fichiers et I/O **2**
0.1 Ouvrir et fermer un fichier 2
0.2 Lire un fichier 3
0.3 Écrire dans un fichier 4
0.4 Le format JSON 4
0.5 Le format CSV 6
0.6 pathlib — Navigation moderne dans les fichiers 7
0.7 Cas pratique — Gestionnaire de notes 8
0.8 Ce qui m'a le plus surpris dans ce chapitre 9
0.9 Résumé du Chapitre 18 10

Chapitre 18 — Fichiers et I/O

À retenir

Un programme qui ne peut pas **lire** et **écrire** des données est condamné à recommencer à zéro à chaque exécution. Les fichiers permettent de **persister** les données — les sauvegarder entre les sessions. Ce chapitre couvre la lecture et l'écriture de fichiers texte, les formats structurés JSON et CSV, et la navigation dans le système de fichiers avec `pathlib`. C'est le pont entre votre programme et le monde réel.

Ouvrir et fermer un fichier

"r"	"w"	"a"	"x"
Lecture seule	Écriture (écrase)	Ajout en fin	Création exclusive
Crée si absent : **Non**	Crée si absent : **Oui**	Crée si absent : **Oui**	Crée si absent : **Oui**
Écrase : —	Écrase : **Oui**	Écrase : **Non**	Écrase : —

Ouvrir un fichier — les bonnes pratiques

```python
#  Methode manuelle     risque d'oublier close()
fichier = open("notes.txt", "w")
fichier.write("Bonjour Python\n")
fichier.close()    # peut etre oublie si exception !

# Avec with     fermeture AUTOMATIQUE et garantie
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Bonjour Python\n")
    f.write("Chapitre 18     Fichiers\n")
# f est ferme automatiquement ici

# Modes combines
# "r+"      lecture ET ecriture (fichier doit exister)
# "w+"      lecture ET ecriture (ecrase si existe)
# "a+"      lecture ET ajout
# "rb"      lecture binaire
# "wb"      ecriture binaire

# L'encodage UTF-8 est crucial pour les accents
with open("texte.txt", "w", encoding="utf-8") as f:
    f.write("Cafe, creme brulee, naif\n")
```

```python

# Toujours preciser l'encodage pour la portabilite
with open("texte.txt", "r", encoding="utf-8") as f:
    contenu = f.read()
    print(contenu)
```

Lire un fichier

read(), readline(), readlines()

```python
# Creer un fichier de test
with open("etudiants.txt", "w", encoding="utf-8") as f:
    f.write("Alice,15,Paris\n")
    f.write("Bob,18,Lyon\n")
    f.write("Charlie,12,Marseille\n")
    f.write("Diana,9,Bordeaux\n")
    f.write("Eve,17,Toulouse\n")

# read()      lire TOUT le fichier d'un coup
with open("etudiants.txt", "r", encoding="utf-8") as f:
    tout = f.read()
    print(f"Contenu complet ({len(tout)} caracteres) :")
    print(tout)

# readline()      lire UNE ligne     la fois
with open("etudiants.txt", "r", encoding="utf-8") as f:
    ligne1 = f.readline()    # "Alice,15,Paris\n"
    ligne2 = f.readline()    # "Bob,18,Lyon\n"
    print(repr(ligne1))
    print(repr(ligne2))

# readlines()      lire TOUTES les lignes dans une liste
with open("etudiants.txt", "r", encoding="utf-8") as f:
    lignes = f.readlines()
    print(f"{len(lignes)} lignes trouvees")
    for ligne in lignes:
        print(ligne.strip())    # strip() supprime \n

#  MEILLEURE PRATIQUE      iterer directement sur le fichier
# Efficace en memoire : lit ligne par ligne
with open("etudiants.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        prenom, note, ville = ligne.strip().split(",")
        print(f"{prenom:10} | {note:3} | {ville}")
```

Écrire dans un fichier

write(), writelines(), print() vers fichier

```python
notes = [
    ("Alice",   15, "Admis"),
    ("Bob",     18, "Mention TB"),
    ("Charlie", 12, "Admis"),
    ("Diana",    9, "Recale"),
]

# write()      ecrire une chaine
with open("resultats.txt", "w", encoding="utf-8") as f:
    f.write("=== RESULTATS EXAMENS ===\n\n")
    for prenom, note, mention in notes:
        ligne = f"{prenom:<12} {note:3}/20  {mention}\n"
        f.write(ligne)
    f.write(f"\nTotal : {len(notes)} etudiants\n")

# writelines()      ecrire une liste de chaines
# (n'ajoute PAS de \n automatiquement !)
with open("prenoms.txt", "w", encoding="utf-8") as f:
    lignes = [f"{prenom}\n" for prenom, _, _ in notes]
    f.writelines(lignes)

# Utiliser print() avec file= pour ecrire
with open("log.txt", "w", encoding="utf-8") as f:
    print("Demarrage du programme", file=f)
    print(f"Nombre d'etudiants : {len(notes)}", file=f)
    for prenom, note, mention in notes:
        print(f"  {prenom}: {note}/20", file=f)

# Mode "a"      ajouter sans ecraser
with open("log.txt", "a", encoding="utf-8") as f:
    from datetime import datetime
    f.write(f"\n[{datetime.now():%d/%m/%Y %H:%M}] Session
        terminee\n")
```

Le format JSON

JSON (JavaScript Object Notation) est le format universel d'échange de données structurées.

json — sérialiser et désérialiser

```python
import json

# Donnees Python      JSON (serialisation)
profil = {
```

```python
    "prenom":   "Alice",
    "age":      25,
    "ville":    "Paris",
    "langages": ["Python", "JavaScript", "SQL"],
    "actif":    True,
    "score":    98.5
}

# json.dumps()      convertir en chaine JSON
chaine_json = json.dumps(profil, ensure_ascii=False, indent=2)
print(chaine_json)

# json.dump()      ecrire dans un fichier
with open("profil.json", "w", encoding="utf-8") as f:
    json.dump(profil, f, ensure_ascii=False, indent=2)

print(" Profil sauvegarde dans profil.json")

# JSON      Donnees Python (d serialisation)
# json.loads()      depuis une chaine
chaine = '{"nom": "Bob", "age": 30, "actif": true}'
donnees = json.loads(chaine)
print(donnees["nom"])     # Bob
print(type(donnees))      # <class 'dict'>

# json.load()      depuis un fichier
with open("profil.json", "r", encoding="utf-8") as f:
    profil_lu = json.load(f)

print(profil_lu["prenom"])       # Alice
print(profil_lu["langages"])     # ['Python', 'JavaScript', 'SQL
    ']
print(profil_lu["actif"])        # True

# Sauvegarder une liste de donnees
etudiants = [
    {"prenom": "Alice", "note": 15, "ville": "Paris"},
    {"prenom": "Bob",   "note": 18, "ville": "Lyon"},
]

with open("etudiants.json", "w", encoding="utf-8") as f:
    json.dump(etudiants, f, ensure_ascii=False, indent=2)
```

À retenir

JSON ne supporte que : `str`, `int`, `float`, `bool`, `None`, `list`, `dict`. Les `datetime`, les `set`, les objets personnalisés ne sont **pas sérialisables** directement. Pour les convertir, il faut un encodeur personnalisé ou les transformer en types supportés avant la sérialisation.

Le format CSV

CSV (Comma-Separated Values) est le format standard pour les données tabulaires.

csv — lire et écrire des tableaux

```python
import csv

# Ecrire un CSV
etudiants = [
    ["Prenom", "Note", "Mention", "Ville"],
    ["Alice",   15,    "Bien",    "Paris"],
    ["Bob",     18,    "TB",      "Lyon"],
    ["Charlie", 12,    "AB",      "Marseille"],
    ["Diana",    9,    "Recal ",  "Bordeaux"],
]

with open("etudiants.csv", "w", newline="",
          encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(etudiants)

print("CSV cree")

# Lire un CSV
with open("etudiants.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    en_tete = next(reader)   # sauter l'en-tete
    print(f"Colonnes : {en_tete}")
    for ligne in reader:
        prenom, note, mention, ville = ligne
        print(f"  {prenom:10} {note:3}  {mention:8}  {ville}")

# DictWriter     ecrire avec des noms de colonnes
with open("notes.csv", "w", newline="",
          encoding="utf-8") as f:
    champs = ["prenom", "note", "ville"]
    writer = csv.DictWriter(f, fieldnames=champs)
    writer.writeheader()
    writer.writerow({"prenom": "Alice", "note": 15, "ville": "
        Paris"})
    writer.writerow({"prenom": "Bob",   "note": 18, "ville": "
        Lyon"})

# DictReader     lire avec acces par nom de colonne
with open("notes.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for ligne in reader:
        print(f"{ligne['prenom']} a eu {ligne['note']}/20")
```

Erreur fréquente

L'argument `newline=""` est **obligatoire** lors de l'ouverture d'un fichier CSV en écriture. Sans lui, Python ajoute des lignes vides supplémentaires sur Windows (
`r`
`n` vs
`n`). C'est une subtilité de compatibilité multiplateforme que j'ai découverte à mes dépens.

pathlib — Navigation moderne dans les fichiers

pathlib — chemins orientés objet

```python
from pathlib import Path

# Creer un chemin      portable Windows/Linux/Mac
dossier  = Path("mon_projet")
fichier  = Path("mon_projet") / "data" / "notes.csv"
absolu   = Path.home() / "Documents" / "python"

print(fichier)           # mon_projet/data/notes.csv
print(fichier.name)      # notes.csv
print(fichier.stem)      # notes
print(fichier.suffix)    # .csv
print(fichier.parent)    # mon_projet/data

# Verifications
print(fichier.exists())     # True/False
print(fichier.is_file())    # True/False
print(fichier.is_dir())     # True/False

# Creer des dossiers
Path("projet/data/raw").mkdir(parents=True, exist_ok=True)

# Lister les fichiers
dossier = Path(".")
for f in dossier.iterdir():
    print(f.name)

# Filtrer par extension
for f in dossier.glob("*.py"):
    print(f"Script Python : {f.name}")

# Recursif      tous les .txt dans les sous-dossiers
for f in dossier.rglob("*.txt"):
    print(f)

# Lire et ecrire directement avec pathlib
chemin = Path("message.txt")
```

```
chemin.write_text("Bonjour Python !\n", encoding="utf-8")
contenu = chemin.read_text(encoding="utf-8")
print(contenu)

# Renommer et deplacer
ancien = Path("ancien_nom.txt")
# ancien.rename(Path("nouveau_nom.txt"))

# Supprimer
# chemin.unlink()         # supprimer un fichier
# Path("dossier").rmdir() # supprimer un dossier vide
```

Cas pratique — Gestionnaire de notes

Système complet de persistence

```
import json
from pathlib import Path
from datetime import datetime

FICHIER_NOTES = Path("notes_etudiants.json")

def charger_notes():
    """Charge les notes depuis le fichier JSON."""
    if not FICHIER_NOTES.exists():
        return {}
    try:
        with open(FICHIER_NOTES, "r", encoding="utf-8") as f:
            return json.load(f)
    except (json.JSONDecodeError, IOError) as e:
        print(f"Erreur de chargement : {e}")
        return {}

def sauvegarder_notes(notes):
    """Sauvegarde les notes dans le fichier JSON."""
    try:
        with open(FICHIER_NOTES, "w", encoding="utf-8") as f:
            json.dump(notes, f, ensure_ascii=False, indent=2)
        print(f" Sauvegarde dans {FICHIER_NOTES}")
    except IOError as e:
        print(f"Erreur de sauvegarde : {e}")

def ajouter_etudiant(notes, prenom, note):
    """Ajoute ou met a jour un etudiant."""
    notes[prenom] = {
        "note": note,
        "date": datetime.now().strftime("%d/%m/%Y"),
        "mention": "TB" if note >= 16 else
```

```
                    "B"  if note >= 14 else
                    "AB" if note >= 12 else
                    "P"  if note >= 10 else "F"
    }
    return notes

def afficher_rapport(notes):
    """Affiche un rapport complet."""
    if not notes:
        print("Aucune note enregistree.")
        return
    print(f"\n{'='*45}")
    print(f"{'RAPPORT DES NOTES':^45}")
    print(f"{'='*45}")
    valeurs = [v["note"] for v in notes.values()]
    for prenom, info in sorted(notes.items()):
        print(f"  {prenom:<12} {info['note']:3}/20  "
              f"{info['mention']:3}  ({info['date']})")
    print(f"{'    '*45}")
    print(f"  Moyenne : {sum(valeurs)/len(valeurs):.1f}/20  "
          f"| Min : {min(valeurs)}  | Max : {max(valeurs)}")


# Utilisation
notes = charger_notes()
notes = ajouter_etudiant(notes, "Alice",   15)
notes = ajouter_etudiant(notes, "Bob",     18)
notes = ajouter_etudiant(notes, "Charlie", 12)
sauvegarder_notes(notes)
afficher_rapport(notes)
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. L'encodage n'est pas optionnel — il est vital. J'ai créé mon premier fichier sans préciser `encoding="utf-8"`. Ça fonctionnait sur mon Mac. Sur Windows, les accents étaient corrompus. Python utilise l'encodage système par défaut — qui varie selon la plateforme. La règle absolue : **toujours préciser `encoding="utf-8"`** lors de l'ouverture d'un fichier texte. Un détail qui peut faire perdre des heures de débogage.

2. Itérer directement sur un fichier — la mémoire intelligente. `f.readlines()` charge **tout** le fichier en mémoire. Pour un fichier de 10 Go, c'est catastrophique. Itérer directement avec `for ligne in f:` lit **ligne par ligne**, sans jamais charger plus qu'une ligne en mémoire. Cette différence entre approche naïve et approche professionnelle peut transformer un programme qui plante en un programme qui tourne.

3. `pathlib` remplace `os.path` — et c'est une révélation. Écrire `Path("projet") / "data" / "notes.csv"` au lieu de `os.path.join("projet", "data", "notes.csv")` n'est pas juste plus court — c'est une **autre façon de penser les chemins**. `Path` est un objet avec des méthodes : `.exists()`, `.read_text()`, `.write_text()`, `.glob()`. On ne manipule plus des chaînes de caractères fragiles — on manipule des chemins typés et intelligents.

Résumé du Chapitre 18

Ce que j'ai appris dans ce chapitre :

- `open(fichier, mode, encoding="utf-8")` — toujours préciser l'encodage.
- `with open(...) as f:` — fermeture automatique et garantie.
- `read()`, `readline()`, itérer sur `f` — trois façons de lire.
- `write()`, `writelines()`, `print(file=f)` — écrire dans un fichier.
- **JSON** : `json.dump()` / `json.load()` — sérialiser des structures.
- **CSV** : `csv.writer` / `csv.DictReader` — données tabulaires.
- **pathlib** : `Path` — chemins orientés objet, portables et expressifs.
- `Path.glob()`, `Path.rglob()` — explorer le système de fichiers.

"Un programme sans persistence est une conversation oubliée. Les fichiers, c'est la mémoire longue terme de vos programmes."